SISTEMAS OPERATIVOS MONOPUESTO (SMR 1.º)

Ejercicios · Scripting básico en Bash

Tipo Ejercicios | **Nivel** 3 · Avanzado | **Tema** 13 | **CM** CM4 | **Duración** 10 h

Herramientas H2 (CMD/PowerShell), H4 (terminal Linux y Bash)

> ℹ️ **Nota.** Esta página acompaña a la explicación del tema. Léela antes de ponerte con los ejercicios.

Índice

- Ejercicios · Scripting básico en Bash
  - Índice
  - Cómo trabajar y cómo entregar
  - Ejercicios
    - Parte 1
    - Parte 2
  - Proyecto final de la unidad
    - Parte A — Script de altas (repaso de usuarios y permisos)
    - Parte B — Copia de seguridad con rotación
    - Parte C — Equivalente en PowerShell
  - Entregables
    - Cómo se corrige el código

Cómo trabajar y cómo entregar

Entorno de trabajo. Escribe los scripts en **Visual Studio Code** desde Windows y ejecútalos en la máquina virtual, usando la **carpeta compartida** que configuraste al instalar Mint.

1. En VS Code, abre tu carpeta de trabajo y crea dentro una carpeta `scripts`.
2. En VirtualBox, con la máquina apagada: **Configuración** → **Carpetas compartidas** → **+**, elige la carpeta `scripts`, nombre `scripts`, marca **Automontar** y **Hacer permanente**.
3. En la máquina virtual, la carpeta aparece en `/media/sf_scripts`. Recuerda que tu usuario tiene que estar en el grupo `vboxsf`.
4. Extensiones recomendadas de VS Code: **Bash IDE** y **ShellCheck**.

> ⚠️ **Los ficheros creados desde Windows pueden tener finales de línea de Windows** (`CRLF`). Bash falla con un mensaje muy confuso: `bad interpreter: /bin/bash^M`. En VS Code, abajo a la derecha, cambia `CRLF` por `LF`. O arréglalo en Linux con `dos2unix script.sh`.

Entrega en Moodle. Un único fichero `.zip` con:

- Los scripts (`ejercicio1.sh`, `ejercicio2.sh`, ...) probados y con permisos de ejecución.
- Un documento con, por cada ejercicio: el enunciado, el código, una captura de la ejecución y una explicación breve de cómo lo has resuelto.

Ejercicios

Parte 1

Ejercicio 1 — Quién soy Un script que limpie la pantalla y muestre qué usuario eres y en qué directorio estás. *Comandos:* `clear`, `whoami`, `pwd`.

Ejercicio 2 — Calendario Pide un mes y un año y muestra el calendario de esa fecha. Comprueba que el mes esté entre 1 y 12 y avisa si no. *Comando:* `cal`.

Ejercicio 3 — ¿Iguales? Pide dos nombres y dice si son iguales o distintos. Amplíalo para que no distinga mayúsculas de minúsculas.

Ejercicio 4 — Calculadora Pide un número, la operación (`+`, `-`, `*`, `/`) y otro número, y muestra el resultado. Usa `case`. Controla la división por cero. *Pista:* `*` *hay que entrecomillarlo o el shell lo expande a la lista de ficheros.*

Ejercicio 5 — Permisos Recibe el nombre de un fichero **como argumento** y dice si tienes permiso de lectura, escritura y ejecución sobre él. Si no existe, sal con `exit 1` y un mensaje.

Parte 2

Ejercicio 6 — Ficheros Recibe **varios** nombres de fichero como argumentos. Para cada uno, comprueba si existe y, si existe, muestra su contenido. Si no existe, avísalo y sigue con el siguiente. *Pista:* `for F in "$@"; do`

Ejercicio 7 — Menú Un programa que limpie la pantalla, muestre un menú y repita hasta que se elija salir:

```
1. Ver el tamaño de una carpeta
2. Listar ficheros
3. Crear un fichero
4. Crear una carpeta
5. Borrar ficheros y carpetas
6. Salir
```

Cada opción debe pedir los datos que necesite y comprobarlos. **La opción 5 debe pedir confirmación** antes de borrar nada. *Comandos:* `du -sh`, `ls -lh`, `touch`, `mkdir -p`, `rm -i`.

Ejercicio 8 — Auditoría de permisos Recorre un directorio y lista los ficheros con permisos excesivos (777 o escritura para «otros»). Une los permisos con los bucles. *Pista:* `find "$DIR" -type f -perm -o+w`

Proyecto final de la unidad

Es el entregable principal de la guía y una pieza del reto ETHAZI.

Parte A — Script de altas (repaso de usuarios y permisos)

Un script que, ejecutado como root, prepare el puesto de un alumno y un profesor:

- Crea los usuarios que reciba como argumentos: `./altas.sh <alumno> <profesor>`
- Crea los grupos `alumnos` y `profesores` si no existen.
- Crea en el `home` del alumno la estructura:

Carpeta	Propietario	Grupo	Requisito
`ejercicios`	el profesor	`profesores`	el alumno **no** puede editar
`examenes`	el profesor	`alumnos`	los profesores no editan; los alumnos sí, sin ser propietarios
`apuntes`	el alumno	`alumnos`	los alumnos tienen todos los permisos

- Crea un enlace simbólico en `/home` que apunte a la carpeta `apuntes` del alumno.
- **Comprueba antes de crear**: si el usuario ya existe, avisa y no falles.
- Registra en un log cada acción realizada.

Parte B — Copia de seguridad con rotación

Un script `copia.sh <origen> <destino>` que:

1. Compruebe que recibe dos argumentos y que el origen existe.
2. Cree en el destino una carpeta o un `.tar.gz` con la **fecha y la hora en el nombre**.
3. Registre inicio, fin, tamaño y resultado en un fichero de log con marca de tiempo.

4. **Rote**: conserve sólo las **7 copias más recientes** y borre las anteriores.
5. Devuelva `exit 0` si todo fue bien y un código distinto si algo falló.
6. Se programe con `cron` para ejecutarse todos los días a las 3:00.

Verificación exigida:

```
crontab -l
ls -lh /ruta/destino
tail -n 20 /ruta/del/log
```

> 💡 **Comprueba la rotación de verdad**: ejecuta el script nueve veces seguidas cambiando la fecha del nombre a mano si hace falta, y demuestra con capturas que sólo quedan siete copias. «Debería funcionar» no es una comprobación.

Parte C — Equivalente en PowerShell

Los ejercicios 5 y 7 y la parte B, en PowerShell, sobre la máquina virtual de Windows, programados con el **Programador de tareas** en lugar de con cron.

Entregables

Parte 1 — Tarea Moodle: «Tres scripts comentados y probados» Ejercicios 1 a 5 en Bash + los ejercicios 5 y 7 en PowerShell. Calificación de 0 a 10 · Se aprueba con 5. *Instrumento: rúbrica de código — funcionamiento, claridad y comentarios.*

Parte 2 — Tarea Moodle: «Script de copia de seguridad automatizado» Ejercicios 6 a 8 + el proyecto final completo (partes A, B y C). Calificación de 0 a 10 · Se aprueba con 5. *Instrumento: prueba práctica T3 + rúbrica de código.*

Cómo se corrige el código

Criterio	Peso
Funciona y hace lo que pide el enunciado	40 %
Controla los errores: argumentos, ficheros que no existen, división por cero, confirmaciones	25 %
Está bien escrito: comillas en las variables, `local` en las funciones, nombres claros, sin código repetido	20 %
Está documentado: cabecera, comentarios que explican el *por qué*, mensaje de uso	15 %

> ℹ️ Un script que funciona pero que se rompe en cuanto le pasas un nombre con un espacio no está terminado. Prueba siempre tus scripts con: ningún argumento, argumentos de más, un fichero que no existe, y un nombre con espacios.
